

NairaPlate scoping document: offline resilience

Status: scoping for review. Nothing is built. It is meant to be argued with before any design is chosen. **Date:** 3 October 2026

1. The problem

Network drops are common in Nigerian kitchens. If the till stops working in the middle of service, staff go back to paper or memory, and those sales never reach the books. That defeats the reason for NairaPlate: money, stock and credit that cannot quietly leak.

An independent review raised this as a P1 blocker for commercial rollout. I agree with the problem. I do not yet know how often drops happen at the kitchens we want to serve, and that number changes how much to build (see Section 8).

2. What exists today (facts from the code)

- **No offline mode.** The FAQ says: "NairaPlate needs an internet connection. We have not built an offline mode yet." There is no service worker and no local queue. The app has a home-screen manifest only.
- **A sale is one server call.** The till calls `create_cash_order`. The database reads the menu price itself, checks the cash adds up, saves the order and its lines, freezes the plate cost, reduces stock, and refuses a cook, purchaser or visitor. It is atomic: all or nothing.
- **Sign-in needs the server.** A person signs in with business code and PIN. The server checks the PIN and returns a session with a refresh token. I have not confirmed how long that session lasts before it needs the network.
- **Expected cash depends on time.** A shift's expected cash is worked out from sales made between the shift's opening and closing time.
- **Void and refunds are same-day, reasoned, and tied to an order that exists on the server.**

3. Why offline is hard for this product in particular

The whole value of the product is that the server refuses things. Offline, a device acts on its own. Each rule below has to be re-decided.

Rule today	The offline problem
The database sets the price	The device only has a price from its last sync. The price may have changed
Stock is reduced at the sale	Two devices can sell the last plate offline
One open shift per business	A shift could be opened or closed elsewhere while this device is offline
Expected cash uses the sale time	A sale synced after the shift closed would fall outside it
PIN is checked only on the server	Storing PIN data on a device is a security risk

Rule today	The offline problem
Void needs the order to exist	A sale made offline has no server order to void yet
A sale cannot be duplicated	A retry after a dropped connection can send it twice

4. Options

Option A: A "late entry" feature, no offline queue (small)

Paper stays the fallback. After the network returns, a cashier or owner enters the paper sales in a **Late entry** form with the real time they happened. The database accepts it only with a reason, an owner approval, and a marker saying it was keyed in late.

- **Gives:** a clean path from paper to the books, with an audit trail. Works for any outage length.
- **Does not give:** selling through an outage.

Option B: Offline cash sales on an already signed-in device (medium)

The device keeps a cached copy of the menu and prices. If the network drops, a signed-in cashier can keep ringing up **cash and split-with-cash sales only**. Sales wait in a local queue and send when the network returns. No transfer, no credit, no void or refund while offline.

- **Needs:** a unique id on each queued sale so a retry never duplicates; the server re-prices each sale on sync; a rule for a price that has changed; an "occurred at" time kept with the sale; a visible banner and count of sales waiting.
- **Does not give:** signing in while offline.

Option C: Full offline (large, and I do not recommend starting here)

Includes signing in offline, voids and refunds offline, stock and drawer logic on the device. It needs a local security model for PINs, conflict resolution between devices, and a reconciliation step for each ledger. It is a rewrite of the trust model, not an addition.

5. Recommendation

1. **Do Option A first.** It is small, closes the paper gap, and fits the audit model. It also gives the owner a place to correct what happened during an outage.
2. **Then do Option B** for cash sales only, if the field data (Section 8) shows outages are common and long enough.
3. **Leave Option C** until Option B has been used in a real kitchen.

6. Design points for Option B (so they can be challenged now)

1. **Idempotency.** Each queued sale gets a unique `client_sale_id`. The server stores it with the order and refuses a second order with the same id, returning the first.
2. **Re-pricing on sync.** The server prices each queued sale again at sync using the menu price **at the time of the sale** (needs price history by time, which does not exist yet). Where that is not available, the choices are: accept the price on the device; or flag the sale for the owner when the server price differs. I recommend accept-and-flag.
3. **Sale time.** The order stores `occurred_at` (device time, checked against server time at sync) as well as `created_at`. Expected cash then uses `occurred_at`. A device clock that is very

wrong is rejected.

4. **Stock.** Stock is reduced at sync. Stock may go below zero, which the system already allows and alerts on. No attempt to block overselling offline.
5. **Drawer.** Queued sales belong to the shift that was open when they were made. If the shift has since been closed, the close is blocked while there are unsynced sales, or the sales are held for the owner. Needs a rule.
6. **Security.** No PIN or PIN hash is stored on the device. Offline work is allowed only while the existing session is still valid, for a limited time after losing connection (a limit the owner sets). When it runs out, selling stops with a clear message.
7. **Visibility.** A clear banner "Offline: N sales waiting". Sales cannot be silently dropped. If a sale fails at sync, it goes to a "needs attention" list for the owner, never discarded.
8. **Audit.** Each synced sale carries an audit marker that it was made offline, with the delay between the sale and the sync.
9. **Refunds.** Not allowed offline. A sale made offline can be voided or refunded only after it syncs.

7. Risks

- **Duplicate or lost sales** if idempotency or the queue is wrong. This is the main risk and must be proved with airplane-mode tests, not only unit tests.
- **Price disputes** after a price change during an outage.
- **Clock manipulation:** a cashier changes the device clock to move a sale into another shift.
- **Storage:** browsers can clear local data. A queue that holds money records on a phone is fragile. It needs a safe-storage check and a warning when the queue is not saved.
- **Support load:** staff will ask "did my sale go through?" The banner and a sync log must answer that.

8. What I need to know first

1. How often do outages happen at the kitchens you target, and for how long? One real figure per site would change the recommendation.
2. Which devices do cashiers use (phone, tablet, laptop) and which browser?
3. Is a late-entry form (Option A) acceptable as the first deliverable?
4. For a price that changed during an outage: accept the device price and flag it, or reject the sale?
5. How long may a device keep selling offline before it must reconnect?

9. How it would be tested

- Airplane-mode tests on real phones: sell, lose connection mid-sale, close the browser, restore the connection, double-tap, two devices selling the last plate.
- A database rehearsal for idempotency, a changed price, a closed shift, a bad clock and a refused role.
- The UAT scripts gain an offline section with step-by-step instructions.

10. Size (my estimate, not a commitment)

Option A: small. Option B: medium to large, mostly because of the testing. Option C: large. I have not planned effort in days and would not trust a number before the questions in Section 8 are answered.